

FLEXBOX IN CSS

Complete Guide with Examples & Diagrams

All Properties • All Values • Practical Examples & Visual Diagrams

Table of Contents

1. Introduction to Flexbox
2. Flexbox Container Properties
3. Flexbox Item Properties
4. Property Reference Table
5. Practical Examples with Diagrams
6. Common Use Cases
7. Quick Tips & Cheat Sheet

1. Introduction to Flexbox

Flexbox (Flexible Box Layout) is a CSS layout module that provides an efficient way to arrange, distribute, and align elements within a container. It offers a one-dimensional layout method for arranging items in rows or columns with automatic space distribution.

Key Benefits:

- **Responsive:** Automatically adjusts to different screen sizes
- **Simple:** Less code compared to float or positioning
- **Flexible:** Easy to reorder elements and change layout direction
- **Alignment:** Powerful horizontal and vertical alignment
- **Space Distribution:** Automatically distribute available space

2. Flexbox Container Properties

display: flex / inline-flex

Initiates flexbox layout on the container. **flex** makes the container a block-level flex container, while **inline-flex** makes it inline-level.

CS

```
.container { display: flex; }
```

flex-direction

Defines the direction of flex items. **row** (default) arranges items horizontally left-to-right. **row-reverse** reverses the order. **column** arranges items vertically top-to-bottom. **column-reverse** reverses vertical order.

Values: row | row-reverse | column | column-reverse

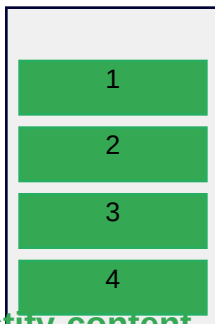
```
.container { flex-direction: column; }
```

Visual Example:

flex-direction: row



flex-direction: column



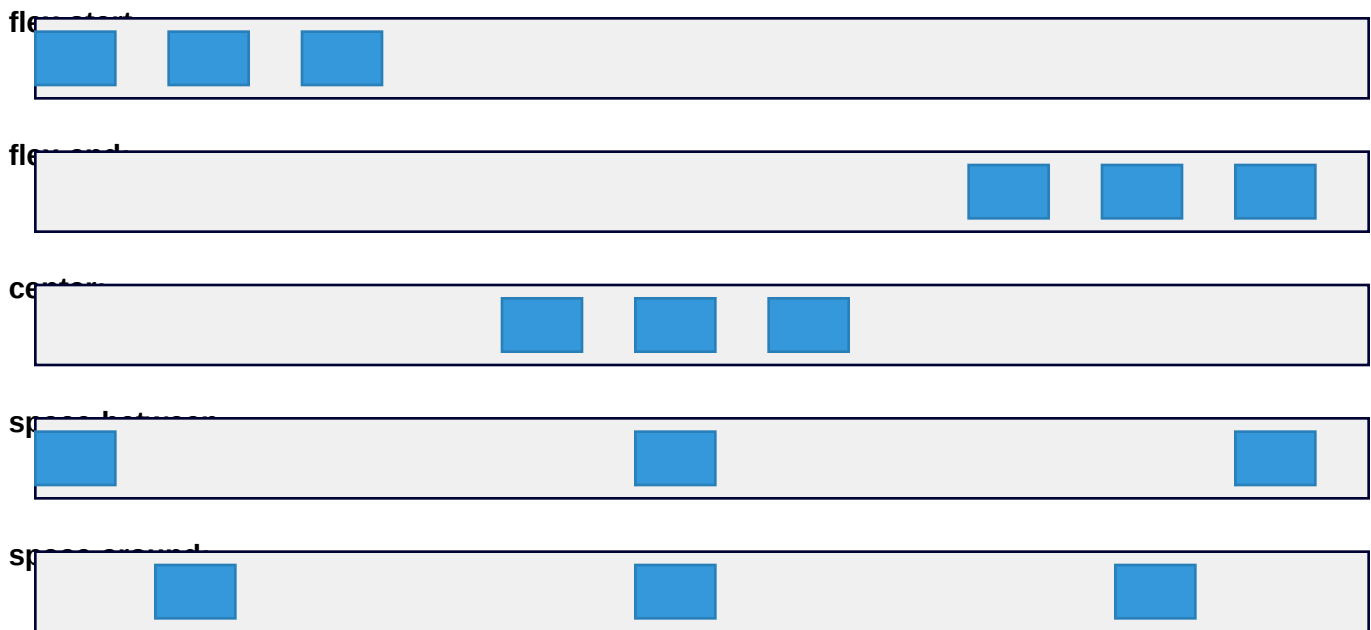
justify-content

Aligns flex items along the main axis (horizontally in row direction). Controls how space is distributed between and around items.

Value	Description
flex-start	Items align to the start
flex-end	Items align to the end
center	Items center along the axis
space-between	Equal space between items
space-around	Equal space around items
space-evenly	Equal space everywhere

```
.container { justify-content: space-between; }
```

justify-content: Visual Diagram



align-items

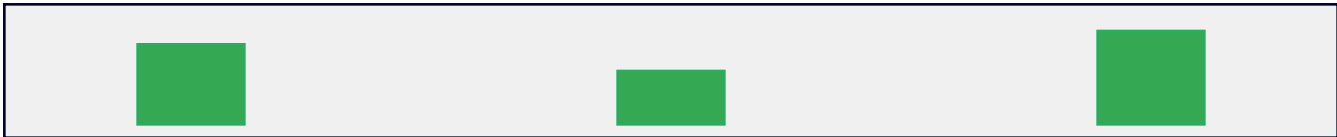
Aligns flex items along the cross axis (vertically in row direction). Controls vertical alignment on a single line.

Value	Description
flex-start	Items align to cross start
flex-end	Items align to cross end
center	Items center on cross axis
stretch	Items stretch to fill (default)
baseline	Items align by baseline

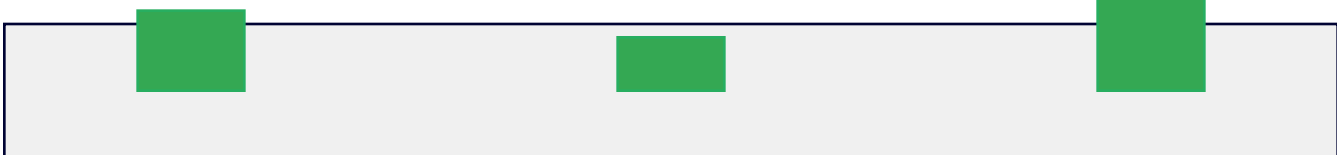
```
.container { align-items: center; }
```

align-items: Visual Diagram

flex-start:



flex-end:



center:



stretch:



flex-wrap

Defines whether flex items should wrap onto multiple lines. **nowrap** keeps all items on one line, **wrap** allows wrapping, **wrap-reverse** wraps in reverse order.

Values: nowrap | wrap | wrap-reverse

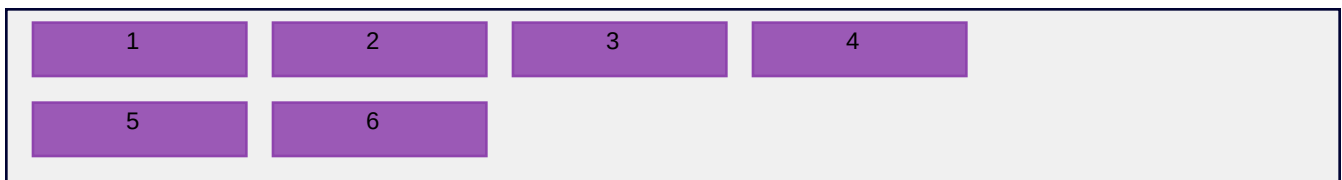
```
.container { flex-wrap: wrap; }
```

flex-wrap: Visual Diagram

flex-wrap: nowrap



flex-wrap: wrap



align-content

Aligns flex lines when items wrap to multiple lines. Only works when **flex-wrap: wrap** is set. Similar to justify-content but for vertical alignment of multiple lines.

Values: flex-start | flex-end | center | space-between | space-around | space-evenly | stretch

```
.container { align-content: center; flex-wrap: wrap; }
```


gap (row-gap, column-gap)

Creates space between flex items. **gap** sets both row and column gap. **row-gap** sets vertical spacing. **column-gap** sets horizontal spacing. Cleaner alternative to using margin.

```
.container { gap: 10px 20px; } /* 10px row-gap, 20px column-gap */
```

flex-flow (shorthand)

Combines flex-direction and flex-wrap into one property. Example: **flex-flow: row wrap** is equivalent to **flex-direction: row; flex-wrap: wrap;**

```
.container { flex-flow: column wrap; }
```

3. Flexbox Item Properties

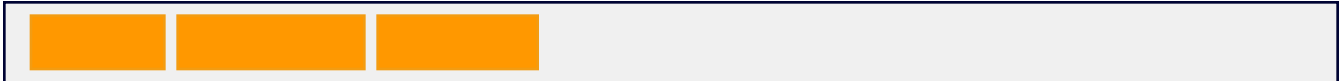
flex-grow

Defines how much a flex item should grow relative to other items when there is extra space. Default is 0 (no growth). Higher values mean more growth.

```
.item { flex-grow: 1; } /* Grows equally with others having flex-grow: 1 */
```

flex-grow: Visual Diagram

Without flex-grow (all items have content width):



With flex-grow: 1 (equal sizes, fills container):



flex-shrink

Defines how much a flex item should shrink relative to other items when there is not enough space. Default is 1. Higher values shrink more.

```
.item { flex-shrink: 0; } /* Won't shrink even if container is small */
```

flex-basis

Defines the default size of a flex item before the remaining space is distributed. Can be a length (px, em) or percentage. Acts like width in row direction, height in column direction.

```
.item { flex-basis: 200px; } /* Base size of 200px */
```

flex (shorthand)

Combines flex-grow, flex-shrink, and flex-basis. Syntax: **flex: grow shrink basis**. Common values: **flex: 1** (equal growth), **flex: 0 0 auto** (no flex)

```
.item { flex: 1 1 200px; } /* Grows/shrinks with 200px base */
```

align-self

Overrides the container's align-items property for individual items. Allows different cross-axis alignment for specific items.

Values: auto | flex-start | flex-end | center | stretch | baseline

```
.item { align-self: flex-end; } /* This item aligns to cross end */
```

order

Changes the visual order of flex items without modifying HTML. Default is 0. Items with lower order appear first. Can be negative.

```
.item:nth-child(1) { order: 3; } .item:nth-child(2) { order: 1; }  
.item:nth-child(3) { order: 2; } /* Visual order: 2nd, 3rd, 1st */
```

4. Complete Property Reference

CONTAINER PROPERTIES

Property	Values	Default
display	flex inline-flex	-
flex-direction	row row-reverse column column-reverse	row
flex-wrap	nowrap wrap wrap-reverse	nowrap
justify-content	flex-start flex-end center space-between space-around space-evenly	flex-start
align-items	flex-start flex-end center stretch baseline	stretch
align-content	flex-start flex-end center space-between space-around space-evenly	stretch
gap	length percentage	0

ITEM PROPERTIES

Property	Values	Default
flex-grow	number	0
flex-shrink	number	1
flex-basis	length percentage auto	auto
flex	<grow> <shrink> <basis>	0 1 auto
align-self	auto flex-start flex-end center stretch baseline	auto
order	integer	0

5. Practical Examples with Diagrams

Example 1: Basic Flex Row Layout

HTML

```
<div class='container'>
  <div class='item'>Item 1</div>
  <div class='item'>Item 2</div>
  <div class='item'>Item 3</div>
</div>
```

CSS

```
.container {
  display: flex;
  flex-direction: row;
  justify-content: space-between;
  gap: 10px;
}
.item {
  flex: 1;
  padding: 20px;
  background: #3498db;
}
```

Result: Creates three equal-width items with space between them on a single row.

Example 2: Navigation Bar Layout

CSS

```
nav {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 10px 20px;
  background: #333;
}
.logo { flex: 0 0 auto; /* Don't shrink */ }
.menu { display: flex; gap: 15px; }
.menu a { color: white; text-decoration: none; }
```

Result: Logo stays at left, navigation links spread on right with equal spacing.

Example 3: Perfect Centering

CSS

```
.container {
  display: flex;
  justify-content: center;
  align-items: center;
  width: 100%;
  height: 400px;
}
.box {
  width: 200px;
  padding: 20px;
}
```

Result: Box is perfectly centered both horizontally and vertically.

Example 4: Card Grid with Wrapping

```
CS<
    .container {
      display: flex;
      flex-wrap: wrap;
      gap: 15px;
      justify-content: center;
    }
    .card {
      flex: 0 0 calc(25% - 12px);
      background: white;
      border-radius: 8px;
      box-shadow: 0 2px 8px rgba(0,0,0,0.1);
    }
  }
```

Result: Cards display in a 4-column grid on desktop, wrapping to fewer columns on smaller screens.

Example 5: Sidebar + Main Content

```
CS<
    .container {
      display: flex;
      gap: 20px;
    }
    .sidebar {
      flex: 0 0 250px; /* Fixed width sidebar */
    }
    .main {
      flex: 1; /* Takes remaining space */
    }
  }
```

Result: Sidebar maintains fixed width while main content stretches to fill available space.

Example 6: Complete Page Layout Diagram



This diagram shows a typical web page layout using flexbox with a header, sidebar, main content, and footer.

6. Common Use Cases

- **Navigation Bars:** Use flex with justify-content: space-between to position logo left and menu items right.
- **Hero Section:** Combine flex with justify-content and align-items center to create centered hero content.
- **Product Cards Grid:** Use flex-wrap with flex: 0 0 calc(percentage) to create responsive card grids.
- **Footer Layout:** Use flex-direction: column or row to arrange footer sections and links.
- **Modal Dialogs:** Center modals perfectly using justify-content and align-items with full viewport height.
- **Mobile Menu:** Use flex-direction: column to stack menu items vertically on mobile devices.
- **Flexbox Form:** Arrange form labels and inputs horizontally or vertically with consistent spacing.
- **Blog Comment Section:** Use flex with nested containers for author image, name, date, and comment text.

Quick Tips & Best Practices

1. Always set **display: flex** on the **container**, not the items.
2. Use **gap** instead of **margin** for spacing between flex items - cleaner and more predictable.
3. **flex: 1** is shorthand for **flex: 1 1 0**; - items grow and shrink equally.
4. **flex-basis: 0** makes items **equal width** regardless of content.
5. **min-width: 0** prevents flex items from **overflow** when using **flex: 1** with long text.
6. Don't forget about **flex-wrap: wrap** when you want items on multiple lines.
7. Use **order** property to **rearrange items visually** without changing HTML order.
8. **align-content** only works with **flex-wrap: wrap** on multiple lines.
9. **justify-content** aligns on **main axis** (horizontal in row, vertical in column).
10. **align-items** aligns on **cross axis** (vertical in row, horizontal in column).

Flexbox Cheat Sheet

Basic Setup:

```
display: flex;  
flex-direction: row; /* default */
```

Center Everything:

```
justify-content: center;  
align-items: center;
```

Space Items Out:

```
justify-content: space-between;  
justify-content: space-around;  
justify-content: space-evenly;
```

Wrap Items:

```
flex-wrap: wrap;  
gap: 15px;
```

Equal Sized Items:

```
flex: 1; /* on items */
```

Fixed + Flexible:

```
flex: 0 0 200px; /* fixed */  
flex: 1; /* flexible */
```

Vertical Stack:

```
flex-direction: column;  
align-items: stretch; /* full width */
```

Responsive:

```
@media (max-width: 768px) {  
  flex-direction: column;  
}
```